

Episode 2: OOO Microarchitecture

Register Rename, Free List & Intra-Group Forwarding

Deep-dive using `hello_uart.s` simulation logs · Superscalar-LEGv8

§1 — The Problem: WAW and RAW Hazards in Superscalar

In a 2-wide superscalar processor, we want to issue two instructions every clock cycle. The repeating pattern in `hello_uart.s` immediately breaks a naive pipeline:

```
ADDI X6, X0, #72      ; X6 = 'H'      ← writes X6
STR  X6, [X9, #0]      ; store X6      ← reads X6 (RAW hazard: must wait for ADDI)
ADDI X6, X0, #101     ; X6 = 'e'      ← writes X6 again (WAW hazard with first ADDI)
STR  X6, [X9, #4]      ; store X6      ← reads X6 (RAW hazard again)
... (repeats 13 more times)
```

Hazard definitions

Hazard	Full name	What causes it	Effect in naive pipeline
RAW	Read After Write	Instruction B reads a register that instruction A has not finished writing yet	B must stall until A completes — kills throughput
WAW	Write After Write	Two instructions both write to the same architectural register	Second write must wait for first to commit — serializes execution
WAR	Write After Read	B writes to a register that A still needs to read	False dependency — does not exist if values are in different physical regs

The core insight
WAW and WAR hazards are FAKE. They only exist because two different values share the same architectural register name (X6). If we give each new value its own physical register, these hazards vanish. RAW hazards are REAL — a value genuinely depends on a previous computation — but they can be managed by tracking physical register readiness rather than stalling the whole pipeline.

How often does this happen in `hello_uart.s`?

The string-building section of `hello_uart.s` writes X6 sixteen times in a row — once per character. In an in-order machine, each `ADDI X6` would have to wait for the previous `STR X6` to complete before it could proceed. That means 16 sequential stalls, one per character, destroying any parallelism. The rename stage eliminates all of them.

§2 — The Solution: Register Renaming

Architectural registers vs physical registers

Your LEGv8 core has 32 architectural registers (X0-X31) that the programmer and ISA see. Inside the hardware, there are 64 physical registers. The first 32 (p0-p31) shadow the architectural registers directly. The remaining 32 (p32-p63) form the rename pool — fresh physical registers available to hold speculative values.

Architectural (what programmer sees)		Physical (what hardware uses)
X0 (XZR)	→	p0 (always 0)
X1	→	p1..p31 (shadow of X1-X31)
X2		
X3	→	p32 (first rename reg)
X4	→	p33
X5	→	p34
X6 ← key	→	... rename pool ...
X9 ← key	→	p59
...	→	p60
X30 (LR)	→	p61
X31	→	p62
		p63

Why 64 physical registers?

The rename pool must be at least as large as the ROB depth. Your ROB holds 32 entries (confirmed in the log — occ peaks at 17 with ROB_ALLOC idx wrapping at 32). With 32 rename registers (p32-p63), you can have up to 32 in-flight writes simultaneously. If the ROB were larger, you'd need more physical registers.

The rename table

The rename table is the core data structure. It is an array indexed by architectural register number (0-31). Each entry holds the physical register number that currently holds the most recent speculative value of that architectural register.

```
// rename_table.sv – simplified model
logic [5:0] map_table [32];          // map_table[arch_reg] = phys_reg
logic [5:0] committed_table [32];  // last committed mapping (for flush recovery)

// At reset: identity mapping – arch X1 → phys p1, etc.
// map_table[0] = p0 (X0=XZR, always 0, never changes)
// map_table[1] = p1 (initial: X1 maps to its shadow physical)
// ...
// map_table[6] = p6 (initial: X6 maps to p6)

// After first ADDI X6, X0, #72 is renamed:
// map_table[6] = p40 ← X6 now points to the newly allocated p40
```

```
// committed_table[6] still = p6 (committed state unchanged until ROB commit)
```

The free list

The free list tracks which physical registers in the rename pool (p32-p63) are available for allocation. It is implemented as a 64-bit bitmask. A 1 means free, 0 means in use. On reset, bits 32-63 are set (all free) and bits 0-31 are clear (those are the shadow registers, not allocatable).

```
// free_list.sv - allocation logic (simplified)
logic [63:0] free_mask; // bit 1 = free, bit 0 = in use
logic [5:0] alloc_ptr; // round-robin pointer into p32-p63

// Allocation: find next free bit starting from alloc_ptr
// for (search = 0; search < 32; search++) {
//     idx = 32 + ((alloc_ptr - 32 + search) % 32);
//     if (free_mask[idx]) { alloc_phys = idx; free_mask[idx] = 0; break; }
// }

// Release (when ROB commits): set free_mask[old_phys] = 1
// e.g. when X6→p36 commits and p40 becomes the new X6:
//     free_mask[36] = 1 ← p36 is now truly dead, return to pool
```

Time	Free list state (p32-p63)	Key observation
Reset	p32-p63 all FREE (32 slots)	32 rename registers available
After Cycle 3 rename	p32,p33,p34,p35 allocated → 28 free	4 in-flight instructions
After Cycle 8 rename	p32-p42 allocated → 21 free	11 instructions in flight
At Cycle 8 COMMIT (rob0,1)	p32,p33 freed → back to 23 free	ROB commits → free list grows
Steady state (Cycle 10+)	~17 slots occupied, 15 free	ROB occupancy ≈ 17 (confirmed by log)
X6 rename sequence	p36→p40→p41→p42→p43... freed as committed	Each ADDI X6 consumes one slot; each commit frees the old one

§3 — The Rename Stage in Hardware

Module interface (rename_stage.sv)

The rename stage sits between Decode and the Reservation Station. It receives up to FETCH_W=2 decoded instructions per cycle and produces renamed instructions with physical register numbers. It has three major submodules:

Submodule	Purpose	Key ports
free_list	Allocates physical registers from p32-p63 pool	alloc_en[2], alloc_phys[2][6], alloc_valid[2]
rename_table	Maps arch registers to physical, handles forwarding	arch_rs1/rs2 in → phys_rs1/rs2 out; rename_en/arch_rd/new_phys_rd for updates

Submodule	Purpose	Key ports
Pipeline regs	Latches renamed outputs for RS/ROB	rename_prs1, rename_prs2, rename_prd, rename_valid, rename_opcode...

Three operations every cycle

- **ALLOCATE**: for each valid instruction with a non-X0 destination, call `free_list` to get a new physical register.
- **MAP**: look up the current physical register for each source register in the rename table.
- **FORWARD**: if instruction slot 0 writes to a register that slot 1 reads, override the table lookup with the just-allocated physical register — the table hasn't been updated yet (that's clocked).

Intra-group forwarding — the critical detail

This is the subtlest part of the rename stage and the most common source of bugs. The rename table is updated on the clock edge. But two instructions are processed combinationaly in the same cycle. If slot 0 writes X6 and slot 1 reads X6, the table still shows the old X6 mapping when slot 1 looks it up. The forwarding logic detects this and overrides:

```
// rename_table.sv – forwarding logic (the key section)
always_comb begin
    for (int i = 0; i < LOOKUP_PORTS; i++) begin
        // Default: read from speculative map table
        phys_rs1[i] = map_table[arch_rs1[i]];
        phys_rs2[i] = map_table[arch_rs2[i]];

        // FORWARDING: check all EARLIER rename ports in same cycle
        for (int j = 0; j < RENAME_PORTS; j++) begin
            if (j < i && rename_en[j] && arch_rd[j] != 5'd0) begin
                // Earlier slot is writing a register this slot reads
                if (arch_rs1[i] == arch_rd[j])
                    phys_rs1[i] = new_phys_rd[j]; // override with fresh phys reg
                if (arch_rs2[i] == arch_rd[j])
                    phys_rs2[i] = new_phys_rd[j]; // override with fresh phys reg
            end
        end
    end
end
```

Decode slot	What rename stage does	Why this matters
Slot 0 (ADDI X6,X0,#72)	Allocates p40 for X6. Writes: arch_rd[0]=x6, new_phys_rd[0]=p40	p40 is not yet in map_table (that update is clocked, happens next cycle)
Slot 1 (STR X6,X9,#0)	Needs to read X6 for rs2. Checks map_table → gets OLD p36.	WITHOUT forwarding: STR would use stale p36 → wrong data!
Slot 1 (forwarding active)	rename_table sees: j=0 < i=1, rename_en[0]=1, arch_rd[0]=x6 == arch_rs2[1]=x6 → phys_rs2[1] = new_phys_rd[0] = p40	Forwarding overrides the stale map_table read with the just-allocated p40

Decode slot	What rename stage does	Why this matters
Result	STR issues with prs2=p40, waits for ADDI to write p40, then executes in correct order	Both instructions can be in the pipeline simultaneously with correct dataflow

Common bug without forwarding

Without intra-group forwarding, STR X6,[X9,#0] would be allocated prs2=p36 (the OLD physical mapping of X6). It would then wait in the Reservation Station for p36 to become ready — but p36 is never written by ADDI X6,X0,#72 (which writes p40). The STR would hang forever, deadlocking the ROB.

§4 — Cycle-by-Cycle Rename Trace (from simulation log)

The following table traces the exact rename decisions your hardware made, extracted directly from the simulation log. Cross-reference with hello_uart.lst to verify each instruction's PC.

Cycle	Instruction	Arch rd	Phys rd (new)	Phys rs1	Phys rs2	What this means
2	ADDI X1, X0, #1	x1	p32	p0	p0	First alloc from rename pool. X0→p0 always.
2	ADDI X5, X0, #16 (LSL imm)	x5	p33	p0	p0	Independent — issued same cycle as above.
3	LSL X1, X1, X5 (→ 0x10000)	x1	p34	p32	p33	Reads p32 (prev X1), p33 (prev X5). RAW tracked.
3	ADDI X2, X1, #4	x2	p35	p34	p0	Reads p34 (new X1). Intra-group forward not needed here (p34 not yet written).
4	ADDI X6, X0, #1 (TX mask)	x6	p36	p0	p0	First allocation for X6.
4	ADDI X9, X0, #2 (base>>12)	x9	p37	p0	p0	Independent. X6 and X9 both ready immediately.
5	ADDI X5, X0, #12	x5	p38	p0	p0	X5 gets new physical reg — p33 now stale for X5.
5	LSL X9, X9, #12 (→0x2000)	x9	p39	p37	p38	★ Reads p37 (prev X9=2), p38 (prev X5=12).
6	ADDI X6, X0, #72 ('H')	x6	p40 ★	p0	p0	X6 renamed again. p36 now dead for X6.
6	STR X6, [X9, #0]	x0 (none)	—	p39	p40 ★	★ INTRA-GROUP FORWARD: reads p40 from slot 0 same cycle!
7	ADDI X6, X0, #101 ('e')	x6	p41	p0	p0	Another rename of X6.

Cycle	Instruction	Arch rd	Phys rd (new)	Phys rs1	Phys rs2	What this means
7	STR X6, [X9, #4]	x0	—	p39	p41	Again reads the just-renamed p41 via forwarding.
8	ADDI X6, X0, #108 ('I')	x6	p42	p0	p0	Pattern continues.
8	STR X6, [X9, #8]	x0	—	p39	p42	p39 stays constant (X9 base never changes in this loop).

Key pattern: X6 renamed 16+ times

Every ADDI X6,X0,#N gets a fresh physical register: p40, p41, p42, p43, p44, p45, p46, p47, p48, p49, p50, p51, p52, p53, p54, p55. Each STR X6,[X9,#N] that follows it reads the correct new physical register via intra-group forwarding. The old physical registers are freed as the ROB commits them in order.

Why X9 stays at p39 for the entire string section

X9 holds 0x00002000 (the scratchpad base address) and is never written again after LSL X9,X9,X5 at PC=0x1C. So map_table[9] = p39 for the entire character-writing loop. Every STR X6,[X9,#N] reads prs1=p39 — that one physical register holds the stable base address for all 16 stores.

§5 — ROB Interaction and In-Order Commit

The Reorder Buffer (ROB) tracks all in-flight instructions in program order. Instructions are allocated at the tail (rename stage) and committed at the head (oldest instruction). Commit only happens when the head instruction's result is ready AND all older instructions have already committed.

ROB snapshot — Cycles 8-17

RO B idx	PC	Instruction	Arch rd	Phys rd	State	Notes
0	0x00	ADDI X1,X0,#1	x1	p32	COMMIT ✓	First to commit (Cycle 8)
1	0x04	ADDI X5,X0,#16	x5	p33	COMMIT ✓	
2	0x08	LSL X1,X1,X5	x1	p34	COMMIT ✓	Cycle 11
3	0x0C	ADDI X2,X1,#4	x2	p35	COMMIT ✓	Cycle 14
4	0x10	ADDI X6,X0,#1	x6	p36	COMMIT ✓	Cycle 14 — p36 freed
5	0x14	LSL X9,X9,#12	x9	p37	COMMIT ✓	Cycle 15

ROB idx	PC	Instruction	Arch rd	Phys rd	State	Notes
6	0x18	ADDI X5,X0,#12	x5	p38	COMMIT ✓	Cycle 15
7	0x1C	LSL X9,X9,X5	x9	p39	COMMIT ✓	Cycle 16 — X9=0x2000
8	0x20	ADDI X6,X0,#72	x6	p40	COMMIT ✓	Cycle 16 — p36 freed
9	0x24	STR X6,[X9,#0]	x0	p0	COMMIT ✓	Cycle 17 — MEM write 0x2000=0x48
10	0x28	ADDI X6,X0,#108	x6	p41	IN-FLIGHT	Waiting for commit
11	0x2C	STR X6,[X9,#8]	x0	p0	IN-FLIGHT	
...	...	...	...	...	...	Pipeline stays ~17 entries deep

What commit does to the rename table

```
// When ROB commits rob=4 (ADDI X6,X0,#1 → wrote p36, PC=0x10):
// 1. Architectural register file: ARF[x6] = PRF[p36] (write-through)
// 2. committed_table[6] = p36 (record last committed physical for X6)
// 3. free_list: the PREVIOUS committed physical for X6 is freed
//    (what was committed_table[6] before this? p6 – the initial shadow)
//    → free_mask[6] could be set if p6 ≠ p0..31... actually p6 is a shadow
//    and NOT in the rename pool, so it is NOT freed.
// 4. When rob=8 commits (ADDI X6,X0,#72 → wrote p40):
//    previous committed phys for X6 was p36 → free_mask[36] = 1 ← p36 freed!
// This is why the free list stays healthy: each commit frees the old mapping.
```

ROB occupancy = 17 in steady state

The simulation log shows occ=17 consistently from Cycle 10 onward. This means the processor has 17 instructions simultaneously in-flight between fetch and commit. It is issuing 2 per cycle and committing 2 per cycle in steady state — exactly the 2-wide superscalar throughput it was designed for.

§6 — Branch Misprediction and Pipeline Flush

hello_uart.s contains a backward branch: B send_string at PC=0x000000C0. This is the main send loop — after advancing the pointer and writing a character to UART, it jumps back to send_string (PC=0x00000098) to fetch the next character. The branch predictor did not predict this correctly.

Event	Detail
Branch instruction	B send_string at PC=0x000000C0 (opcode 0x20, imm=-40 words = -160 bytes)
Predicted target	PC fetching speculatively ahead: 0xC8, 0xD0, 0xD8... (wrong — all NOPs past end of program)
Actual target	$0x000000C0 + (-40 \times 4) = 0x000000C0 - 160 = 0x00000020...$ wait – see log: target=0x00000098

Event	Detail
Log evidence	[BRANCH_TAKEN] PC=000000C0 → 00000098 (mispredict=1) at Cycle 30
Target 0x98 = what?	send_string label in hello_uart.s (the LDR X4,[X3,#0] that starts the send loop)
Flush effect	Pipeline flushed at Cycle 30. Fetch redirected to 0x98. In-flight speculative instructions after the branch are squashed.
Recovery	Rename table reverts to committed state. Free list reclaims physical registers allocated to squashed instructions.
Cost	~4 cycles of wasted work (cycles 27-30 fetched wrong instructions). This is why branch predictors matter in OOO cores.

What flush_pipeline does to the rename stage

```
// rename_stage.sv – flush behavior
end else if (flush_pipeline) begin
    rename_valid <= '0;    // squash all in-flight renamed instructions
end

// rename_table.sv – on flush, map_table reverts to committed_table
// (this happens in the rename_table module – not shown explicitly in
// the code above, but implied by committed_table being maintained
// as a separate shadow of the architecturally committed state)

// free_list – physical registers allocated to squashed instructions
// are returned to the free list because they were never committed.
// The commit path never fired for those instructions.
```

Interview-ready explanation of precise exceptions via ROB

The ROB guarantees that even though instructions execute out-of-order, they commit in program order. If a branch misprediction is detected, the ROB squashes all instructions after the branch (they haven't committed, so no architectural state has changed). The rename table reverts to the committed_table snapshot. The free list reclaims the physical registers those squashed instructions held. The pipeline restarts from the correct target address. This is called precise exception support — the architectural state is always consistent with in-order execution up to the point of any exception or mispredict.

§7 — UART Output: Proof the Pipeline is Correct

The simulation log's [UART TX] lines are the ground truth. If the OOO core, rename stage, ROB commit, and LSU memory ordering are all working correctly, the characters must appear in program order regardless of out-of-order execution. They do:

Cycle	UART TX	Hex	MEM addr written	Pipeline state at this point
20	'H'	0x48	0x00002000	ROB commits rob=17 (STR X6,[X9,#0]). 17 instructions in-flight simultaneously.
23	'e'	0x65	0x00002004	Comma-pipeline running smoothly. Branch (B send_string) not yet executed.
25	'l'	0x6C	0x00002008	

Cycle	UART TX	Hex	MEM addr written	Pipeline state at this point
26	'l'	0x6C	0x0000200C	
29	'o'	0x6F	0x00002010	BRANCH resolved: B at 0xC0 → 0x98 (send_string). mispredict=1 → flush.
31	','	0x2C	0x00002014	Pipeline recovering from branch. New instructions fetching from 0x98.
33	' '	0x20	0x00002018	
39	'L'	0x4C	0x0000201C	
42	'E'	0x45	0x00002020	
44	'G'	0x47	0x00002024	
46	'v'	0x76	0x00002028	
48	'8'	0x38	0x0000202C	
50	'!'	0x21	0x00002030	
52	'\r'	0x0D	0x00002034	

What this proves

The UART characters arrive in the correct order: H, e, l, l, o, comma, space, L, E, G, v, 8, !, CR, LF. This means: (1) The rename stage correctly tracked 16 successive writes to X6 without any value being lost or mixed up. (2) Intra-group forwarding correctly fed each STR the right physical register. (3) The ROB committed stores to scratchpad in program order. (4) The send loop correctly loaded from scratchpad and wrote to the UART register. (5) The branch back to send_string (after mispredict recovery) continued with the correct pointer value in X3.

The MEM write sequence confirms store ordering

```
// Scratchpad writes extracted from simulation log:
Cycle 19: [MEM] we:1 addr=00002000 data=00000048 // 'H' stored first
Cycle 22: [MEM] we:1 addr=00002004 data=00000065 // 'e' stored second
Cycle 24: [MEM] we:1 addr=00002008 data=0000006c // 'l'
Cycle 26: [MEM] we:1 addr=0000200c data=0000006c // 'l'
Cycle 28: [MEM] we:1 addr=00002010 data=0000006f // 'o'
Cycle 30: [MEM] we:1 addr=00002014 data=0000002c // ','
// ... all 16 characters in order, despite OOO execution

// Then the load loop reads them back:
Cycle 34: [MEM] we:0 addr=00002000 data=00000048 // LDR reads 'H' → UART TX
```

§8 — Key Takeaways & Interview Questions

Summary of what the rename stage does in one sentence

Every time an instruction writes to an architectural register, the rename stage gives it a brand-new physical register from a free pool, so that two instructions writing to the same architectural name never interfere — eliminating WAW and WAR hazards and enabling out-of-order issue.

Questions you will be asked

'What is register renaming and why do we need it?'

Register renaming decouples the architectural register file (what the ISA defines) from the physical register file (what the hardware implements). We need it because superscalar OOO processors want to issue multiple instructions simultaneously, but sequential code reuses architectural registers constantly — creating false dependencies (WAW, WAR) that would force serialization. By giving each new value its own physical register, we eliminate false dependencies. Only true data dependencies (RAW) remain, and those are handled by the reservation station waiting for the producing instruction's physical register to become ready.

'What is a free list?'

A free list is the hardware structure that tracks which physical registers are available for allocation. In your implementation it is a 64-bit bitmask (one bit per physical register). The rename stage consults it every cycle to find available registers. When an instruction commits and its result is superseded by a newer write to the same architectural register, the old physical register is returned to the free list. The round-robin allocation pointer prevents thrashing on the same registers.

'What is intra-group forwarding?'

In a 2-wide superscalar, two instructions are renamed in the same cycle. If the first instruction writes to X6 and the second reads X6, the rename table hasn't been updated yet (that update is clocked). Without forwarding, the second instruction would read the stale physical register for X6. Intra-group forwarding is a combinational bypass in the rename table that detects this same-cycle write-read pattern and overrides the table lookup with the just-allocated physical register. Without it, every ADDI-STR pair in `hello_uart.s` would deadlock.

'How does the ROB ensure in-order commit?'

The ROB is a circular FIFO. Instructions are allocated at the tail in program order (by the rename stage) and committed at the head. An instruction can only commit when it is at the head AND its result is marked ready (the execution unit has written back). Even if instructions execute out of order (the middle of the ROB might be ready before the head), they wait until they are the oldest unretired instruction. This guarantees that architectural state — and memory stores — are updated in program order, enabling precise exceptions and correct memory ordering.

'What happens on a branch mispredict?'

When the branch unit resolves a branch and finds the predicted path was wrong: (1) A flush signal is asserted. (2) The rename stage squashes all in-flight renamed instructions — they never committed, so no architectural state was changed. (3) The rename table reverts to the `committed_table` snapshot, which reflects the last in-order committed state. (4) The free list reclaims the physical registers allocated to squashed instructions. (5) The fetch unit is redirected to the correct branch target. (6) Execution restarts from the correct address. The cost is the number of cycles from when the branch was fetched to when the mispredict was detected — in your core, roughly 4-6 cycles.